In [32]:
```
import pandas as pd

df = pd.read_csv('../data/MergedData.csv')
df = df[df['crime'] != 'larceny theft']

# Normalize district names to title case and keep only unique districts
df['district'] = df['district'].str.title()
unique_districts = df['district'].unique()
df = df[df['district'].isin(unique_districts)]

#We removed larceny theft so we only had 12 focus crimes.
#We fixed an error in our data where the districs where duplicated



#In order ro run this notebook you will need to have the following libraries ins
#pandas
#matplotlib
#numpy
#scipy
```

Introduction

This assignment has been made by Christian, Cemile & Gustav as a part of the 2026 course in Social Data Analysis and Visualization

In this assignment we used a combined SF crime dataset from 2003 to present, our exact dataset can be found within the data.zip file here on Github.

The focus crimes we have chosen are

- Assault
- Burglary
- Gambling
- Drug offense
- Motor vehicle theft
- Fraud
- Missing Person
- Prostitution
- Weapons offense
- Disorderly conduct
- Robbery
- Vandalism

Assignment 1.1: Temporal Overview

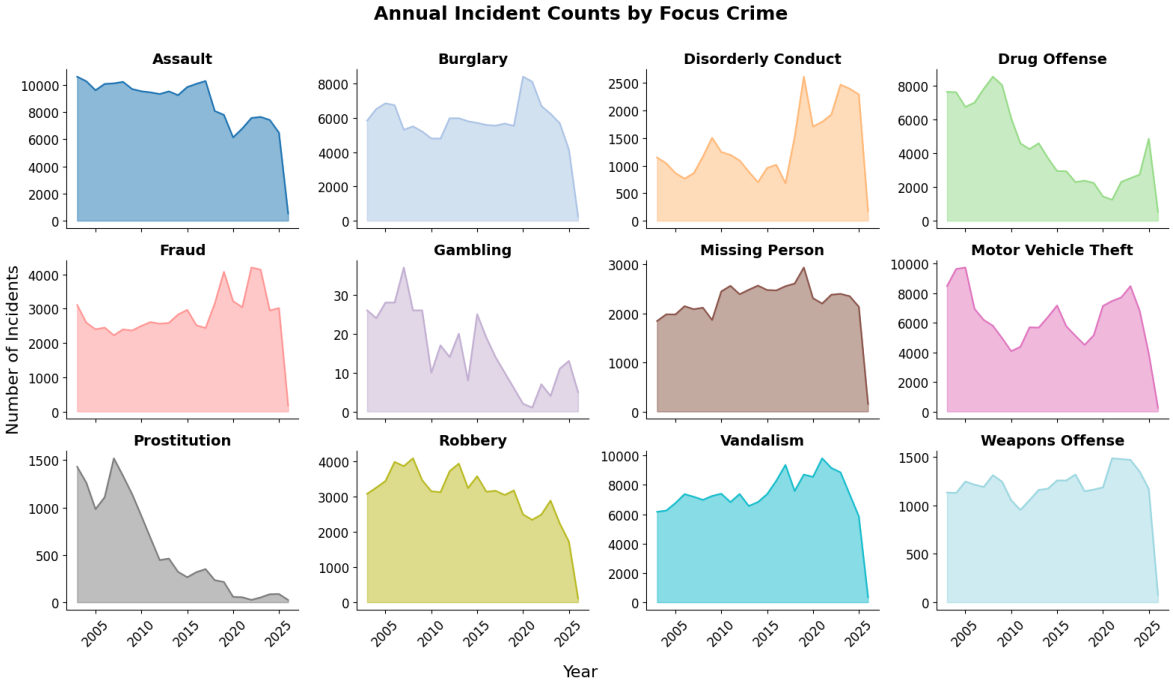
Let's start by getting the big picture of how crime has changed over the full 20+ year period covered by your combined dataset.

Draws from: Week 1, Exercises 4.1 and 4.2 — applied here to the full merged dataset (2003–present) rather than just 2018–present.

> - Using your combined dataset (2003–present), plot the total number of incidents **per year** for each of your Personal Focus Crimes. Display at least the years 2003–2025.
> - Make sure your plot follows good visualization practices: labeled axes, a legend, a descriptive title.
> - Identify and comment on at least two notable features in the plot — for example, long-term trends, sudden drops or spikes, or the impact of COVID-19 in 2020. For each feature, offer a possible explanation.

In [33]:
```
import matplotlib.pyplot as plt
import numpy as np

#Count incidents per year per crime
yearly = df.groupby(["year", "crime"]).size().reset_index(name="incidents")
crimes = sorted(yearly["crime"].unique())
yearly = yearly[yearly["crime"].isin(crimes)]

n_crimes = len(crimes)  # 12
ncols = 4
nrows = 3

fig, axes = plt.subplots(nrows, ncols, figsize=(16, 9), sharey=False)

color_palette = plt.cm.tab20(np.linspace(0, 1, n_crimes))

for i, crime in enumerate(crimes):
    row, col = divmod(i, ncols)
    ax = axes[row, col]
    subset = yearly[yearly["crime"] == crime].sort_values("year")

    ax.plot(subset["year"], subset["incidents"], color = color_palette[i],
            linewidth=1.5)
    ax.fill_between(subset["year"], subset["incidents"], color = color_palette[i
                    alpha=0.50)
    ax.set_title(crime.title(), fontsize=14, fontweight="bold")

    # Only show x-tick labels on the bottom row
    if row == nrows - 1:
        ax.tick_params(axis="x", labelsize=12, rotation=45)
    else:
        ax.tick_params(axis="x", labelbottom=False)

    ax.tick_params(axis="y", labelsize=12)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

# Shared axis labels
fig.supxlabel("Year", fontsize=16)
fig.supylabel("Number of Incidents", fontsize=16)
fig.suptitle("Annual Incident Counts by Focus Crime",
```

```
                fontsize=18, fontweight="bold", y=1.01)

plt.tight_layout()
plt.show()
```



The first interesting feature is noting that in 2020 the amount of gambling fell to almost 0 which could be a mix of different factors. Gambling has moved steadily to online casinos throughout the last decade, a lot of it onto sites registered outside of SFPD or even US jurisdiction. Another factor that seems to compound this trend is Covid 19, where California enforced substantial Stay Home orders.

There is also something to note in the Drug offense plot that President Obama initiated a National Drug Control Strategy (NDCS) in year 2010 - https://www.dhs.gov/archive/news/2010/05/12/2010-national-drug-control-strategy which maybe could have accelerated the downward trend we see in number of drug cases. This trend continued and even got better into Trump's first tenure as president but we see an upwards trajectory under Joe Bidens rule.

Assignment 1.2: Crime Profiles by Police District

Different parts of the city have very different crime patterns. Here we quantify that using conditional probabilities.

Draws from: Week 3, Exercises 2.1 and 2.2.

> - For each police district in your dataset, compute the **conditional crime profile**: for each of your Personal Focus Crimes, calculate
>
> $$r(\text{crime}, \text{district}) = \frac{P(\text{crime} \mid \text{district})}{P(\text{crime})}$$

A value above 1 means that crime type is *over-represented* in that district relative to the city-wide average; below 1 means it is *under-represented*.

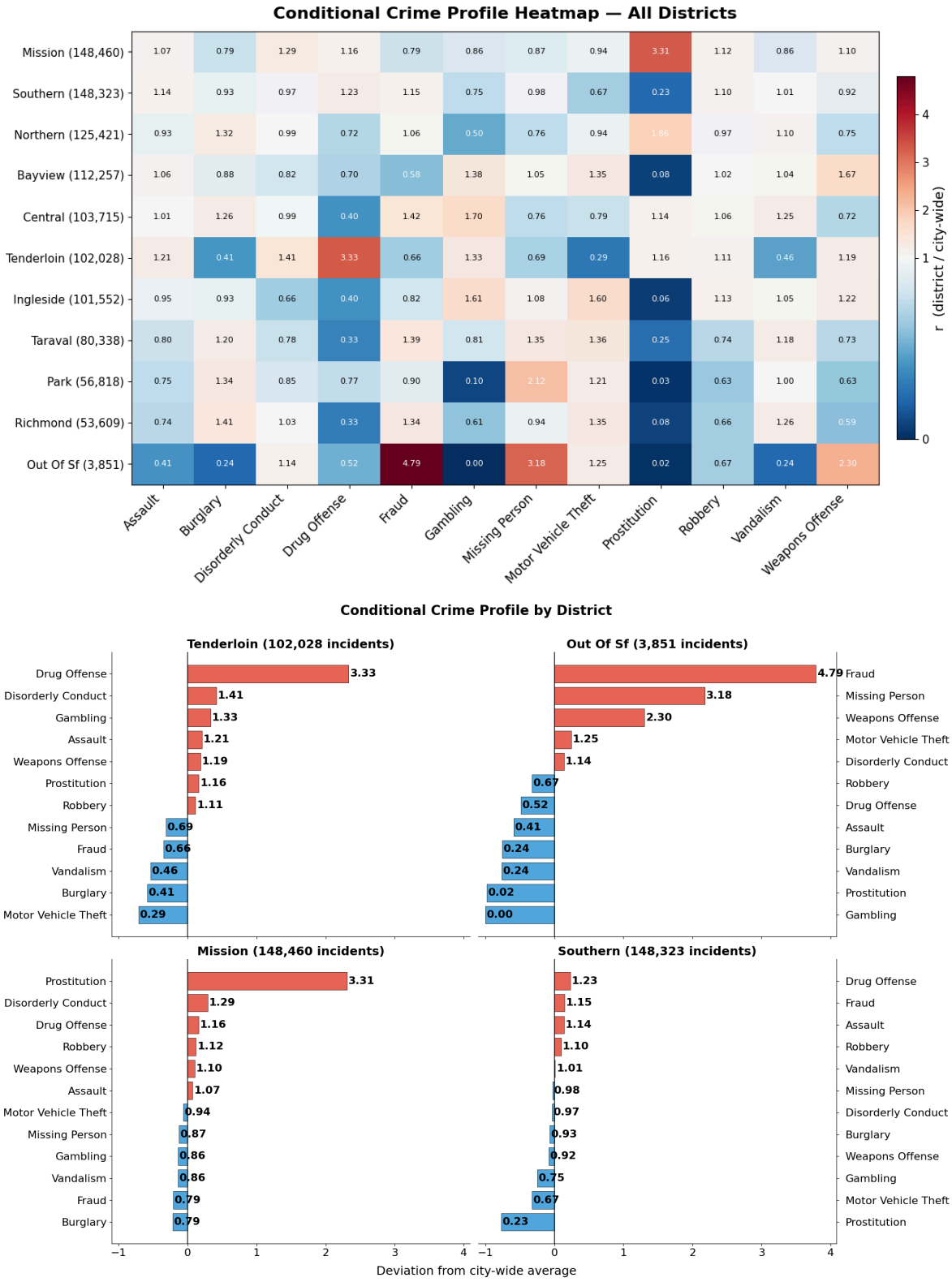
- Visualize these ratios in a way that makes it easy to compare across both districts and crime types. (Simple barcharts are fine, but you may also use ideas from more complex visualization techniques, for example, a heatmap could work well here, but you're free to choose another format if you can justify it.)

- Pick **one district** whose profile stands out to you. Describe the pattern and offer an explanation for why that district looks the way it does. Are there geographic, demographic, or other factors that might explain it?

In [34]:
```
# Get districts sorted by total incidents
district_counts = df['district'].value_counts()
districts = district_counts.index.tolist()

# We use our focus crimes again.
crimes = sorted(df['crime'].unique())

# Compute conditional crime profile
crime_district = df.groupby(['district', 'crime']).size().unstack(fill_value=0)
district_totals = crime_district.sum(axis=1)
crime_totals = crime_district.sum(axis=0)
total_incidents = crime_totals.sum()

p_crime_given_district = crime_district.div(district_totals, axis=0)
p_crime = crime_totals / total_incidents
r = p_crime_given_district.div(p_crime, axis=1)

# ── Heatmap ──────────────────────────────────────────────────────────
import matplotlib.colors as mcolors

r_heatmap = r.loc[districts, crimes]

fig_h, ax_h = plt.subplots(figsize=(14, 0.55 * len(districts) + 2))

# Diverging colormap centered at r=1 (log scale so 0.5 and 2 are equidistant)
norm = mcolors.TwoSlopeNorm(vmin=r_heatmap.values.min(), vcenter=1.0,
                            vmax=r_heatmap.values.max())
im = ax_h.imshow(r_heatmap.values, cmap='RdBu_r', norm=norm, aspect='auto')

ax_h.set_xticks(range(len(crimes)))
ax_h.set_xticklabels([c.title() for c in crimes], rotation=45, ha='right', fonts
ax_h.set_yticks(range(len(districts)))
ax_h.set_yticklabels(
    [f"{d.title()} ({district_counts[d]:,})" for d in districts], fontsize=11
)

# Annotate each cell with the r value
for row_i, district in enumerate(districts):
    for col_j, crime in enumerate(crimes):
        val = r_heatmap.loc[district, crime]
        ax_h.text(col_j, row_i, f'{val:.2f}', ha='center', va='center',
```

```
                  fontsize=8, color='black' if 0.6 < val < 1.8 else 'white')

cbar = fig_h.colorbar(im, ax=ax_h, shrink=0.8, pad=0.02)
cbar.set_label('r  (district / city-wide)', fontsize=13)
cbar.ax.tick_params(labelsize=11)

ax_h.set_title('Conditional Crime Profile Heatmap — All Districts',
               fontsize=16, fontweight='bold', pad=12)
plt.tight_layout()
plt.show()

# ── Bar charts (selected districts, 2x2 grid) ──────────────────────────────
selected_districts = ['Tenderloin', 'Out Of Sf', 'Mission', 'Southern']
all_vals = [r.loc[d] for d in selected_districts]
x_min = min((v.min() - 1) for v in all_vals) - 0.1
x_max = max((v.max() - 1) for v in all_vals) + 0.3

fig, axes = plt.subplots(2, 2, figsize=(20, 14), sharex=True)

for i, district in enumerate(selected_districts):
    row, col = divmod(i, 2)
    ax = axes[row, col]
    vals = r.loc[district].sort_values()

    colors = ['#e74c3c' if v > 1 else '#3498db' for v in vals]

    ax.barh(range(len(vals)), vals.values - 1, color=colors,
            edgecolor='black', linewidth=0.6, alpha=0.85)

    ax.axvline(0, color='black', linewidth=1.2)
    ax.set_yticks(range(len(vals)))
    ax.set_title(f"{district.title()} ({district_counts[district]:,} incidents)"
                 fontsize=18, fontweight='bold')
    ax.set_xlim(x_min, x_max)

    for j, (crime_name, v) in enumerate(vals.items()):
        ax.text(v - 1 + 0.02, j, f'{v:.2f}', va='center', fontsize=16, fontweigh

    ax.spines['top'].set_visible(False)

    # Left column: y-labels on left; right column: y-labels on right
    if col == 0:
        ax.set_yticklabels([c.title() for c in vals.index], fontsize=16)
        ax.yaxis.set_label_position('left')
        ax.yaxis.tick_left()
        ax.spines['right'].set_visible(False)
    else:
        ax.set_yticklabels([c.title() for c in vals.index], fontsize=16)
        ax.yaxis.set_label_position('right')
        ax.yaxis.tick_right()
        ax.tick_params(axis='y', pad=10)
        ax.spines['left'].set_visible(False)

    # Only show x-axis tick labels on the bottom row
    if row == 0:
        ax.tick_params(axis='x', labelbottom=False, labelsize=16)
    else:
        ax.tick_params(axis='x', labelsize=16)

fig.supxlabel('Deviation from city-wide average', fontsize=18)
```

```
fig.suptitle('Conditional Crime Profile by District',
             fontsize=20, fontweight='bold', y=1.01)

plt.tight_layout()
plt.show()
```



Looking at all districts we can see how each one compares to the city-wide average. The heatmap gives a quick overview of the full picture — red cells indicate over-representation and blue cells under-representation relative to the city average — while the bar charts below allow a more detailed per-district look at which specific crimes deviate most.

We also see the amount of total incidents reported so we can get a proper picture of what is going on in each district. The 'Southern' District is for instance very well balanced compared to the city average, and it is also the district with the most crime reported in total, meaning it contributes most to the city-wide baseline.

One could look at the 'Out of SF' bar chart and make a headline about how everybody goes missing or that Fraud is 5 times more likely outside the city, and also that there is no illegal gambling or at most one prostitute working there. But we have to account for the total number of incidents — there are far fewer total crimes recorded in this district than in others, so even a single incident of a rare crime inflates the ratio drastically. The heatmap also makes this clear: 'Out of SF' stands out as a strong outlier in several crime types, but this is an artefact of its very low incident count.

If we want to dive into the 'Tenderloin' District we can however make a robust conclusion: Drug Offense is extremely over-represented here compared to other districts, and since it is one of the districts with the most incidents overall, we can confidently say that drug offenses are genuinely far more likely in Tenderloin than elsewhere in the city. This is clearly visible as a deep red cell in the heatmap. When we research the district this finding is well supported:

"The Tenderloin is a high-crime neighborhood, particularly violent street crime such as robbery and aggravated assault. Graffiti art and tagging are common in the neighborhood. Dealing and use of illicit drugs occurs on the streets."

"Violent acts occur more often here and are generally related to drugs. The area was the scene of escalating drug violence in 2007"

"The first block of Turk Street, between Taylor and Mason, had one of the highest rates of violence and drug activity in San Francisco, according to a survey conducted by the Tenderloin Housing Clinic. On January 31, 2014, parking was banned on both sides of the street in an effort to reduce violence and drug activity. Without parked cars to hide illegal activity, there were fewer loiterers, and a decrease in drug activity"

"In the 2020s, the Tenderloin has experienced many of San Francisco's fentanyl deaths."

Source - https://en.wikipedia.org/wiki/Tenderloin,_San_Francisco

---

Assignment 1.3: Visualizing Distributions

This exercise asks you to recreate several classic plot types from DAOST Chapter 2 using your own crime data — putting visualization theory into practice.

Draws from: Week 3, Exercises 5.2 and 5.3.

> **Part A — Jitter plot**

- Pick one of your Personal Focus Crimes and a suitable time interval (somewhere between a month and 6 months, depending on how common the crime is). Create a jitter plot of the incident times during a single hour (e.g. 13:00–14:00): let time run along the x-axis and add vertical jitter.
- What does the jitter plot reveal about how times are recorded in the dataset? Are incidents clustered at certain minutes (on the hour, half hour, etc.)? What does this tell you about the precision of the data?

Part B — Probability plot

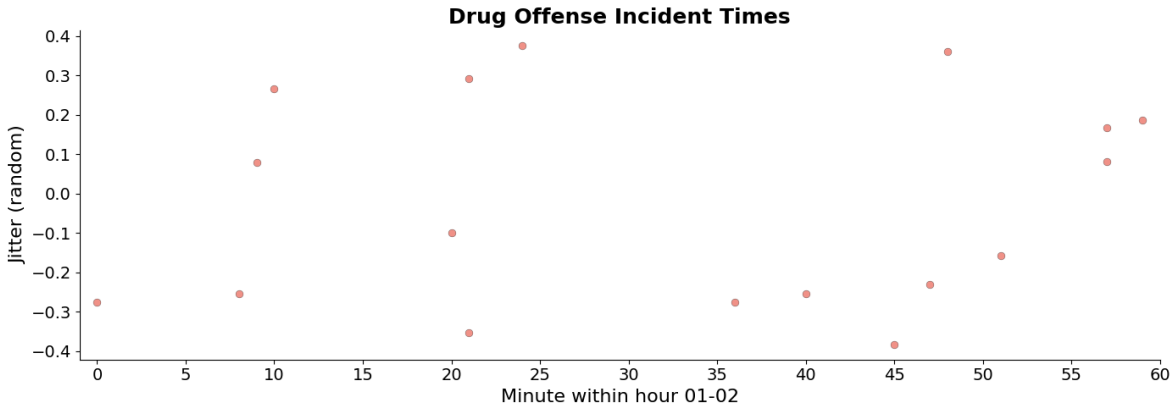
- Using the same geographic data from Part B, create a probability plot (QQ plot) for the latitude distribution of each of your two crime types. (`scipy.stats.probplot` is your friend here.)
- What reference distribution are you comparing against? What would it mean if the points fell exactly on the straight line? Where does the distribution deviate from normal, and what does that deviation tell you about the geography of crime in SF?

Part C — Box plots of time-of-day

- For each of your Personal Focus Crimes, extract the time-of-day of every incident.
- Create box plots showing the time-of-day distribution for all your Personal Focus Crimes side by side.
- What patterns do you see? Are there crimes that happen mostly at night? Mostly during business hours? For crimes that peak late at night, does the box plot handle the wrap-around at midnight well? What goes wrong?
- Above, feel free to use alternatives to box plots — violin plots, swarm plots, or raincloud plots — if you think they reveal more. If you do, briefly explain what the alternative shows that the box plot doesn't.

In [35]:
```
from scipy import stats

# Parse time column to extract hour and minute
df['hour'] = df['time'].str.split(':').str[0].astype(int)
df['minute'] = df['time'].str.split(':').str[1].astype(int)

# Filter: Drug Offense, first 6 months of 2023, from 1Am to 2Am (hour == 1)
drug_hour = df[
    (df['crime'] == 'drug offense') &
    (df['year'] == 2023) &
    (df['month'].isin([1, 2, 3, 4, 5, 6])) &
    (df['hour'] == 1)
].copy()

# X-axis: minute within the hour (0-59)
# Y-axis: random jitter to spread points vertically
np.random.seed(42)
drug_hour['jitter'] = np.random.uniform(-0.4, 0.4, len(drug_hour))
```

```
fig, ax = plt.subplots(figsize=(14, 5))

ax.scatter(drug_hour['minute'], drug_hour['jitter'], alpha=0.6, s=40,
           edgecolor='black', linewidth=0.3, color='#e74c3c')

ax.set_xlabel('Minute within hour 01-02', fontsize=16)
ax.set_ylabel('Jitter (random)', fontsize=16)
ax.set_title('Drug Offense Incident Times',
             fontsize=18, fontweight='bold')
ax.set_xlim(-1, 60)
ax.set_xticks(range(0, 61, 5))
ax.tick_params(axis='both', labelsize=14)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

print(f"Total incidents in this window: {len(drug_hour)}")
```

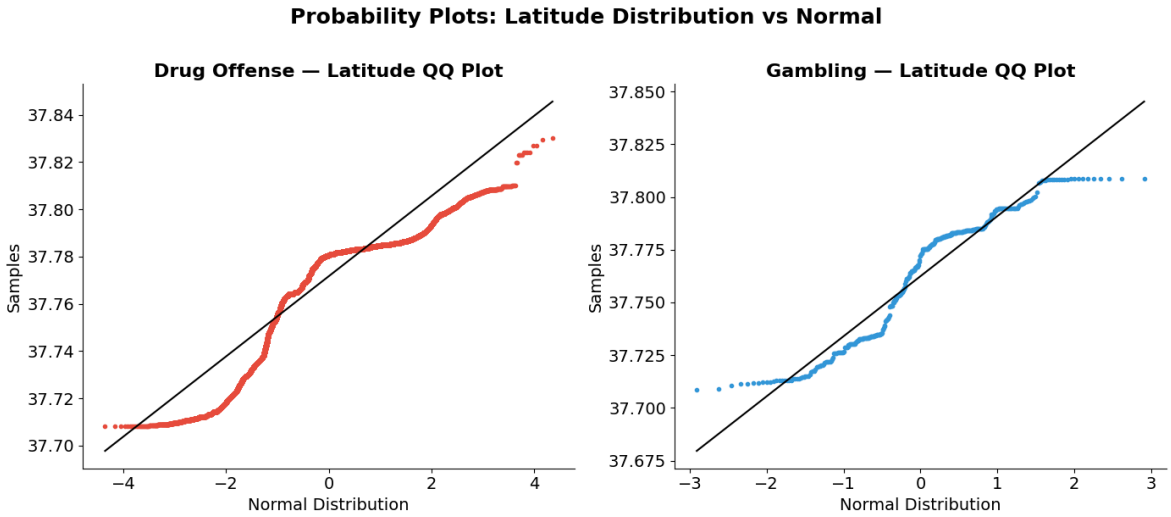



```
Total incidents in this window: 17
```

This plot actually reveals that there is no clear evidence of rounding, its pretty uniformly split within the minute intervals. If any rounding occurs it is to a whole minute and to a whole 5, 10 15 minute.

In [36]:
```
from scipy import stats

# Filter valid latitudes
sf_lat_min, sf_lat_max = 37.70, 37.85
crime1 = df[(df['crime'] == 'drug offense') &
            (df['latitude'] >= sf_lat_min) & (df['latitude'] <= sf_lat_max)]['la
crime2 = df[(df['crime'] == 'gambling') &
            (df['latitude'] >= sf_lat_min) & (df['latitude'] <= sf_lat_max)]['la

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# QQ plot for Drug Offense
ax1 = axes[0]
stats.probplot(crime1, dist="norm", plot=ax1)
ax1.set_title('Drug Offense — Latitude QQ Plot', fontsize=16, fontweight='bold')
ax1.set_xlabel('Normal Distribution', fontsize=14)
ax1.set_ylabel('Samples', fontsize=14)
ax1.tick_params(axis='both', labelsize=14)
ax1.get_lines()[0].set_markersize(3)
ax1.get_lines()[0].set_color('#e74c3c')
```

```
ax1.get_lines()[1].set_color('black')

# QQ plot for Gambling
ax2 = axes[1]
stats.probplot(crime2, dist="norm", plot=ax2)
ax2.set_title('Gambling — Latitude QQ Plot', fontsize=16, fontweight='bold')
ax2.set_xlabel('Normal Distribution', fontsize=14)
ax2.set_ylabel('Samples', fontsize=14)
ax2.tick_params(axis='both', labelsize=14)
ax2.get_lines()[0].set_markersize(3)
ax2.get_lines()[0].set_color('#3498db')
ax2.get_lines()[1].set_color('black')

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

fig.suptitle('Probability Plots: Latitude Distribution vs Normal',
             fontsize=18, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



We compare Drug Offense and Gambling latitude distributions against a normal reference. If incidents were spread uniformly across the city, the points would follow the diagonal line closely. Both plots deviate clearly from this, but in different ways.

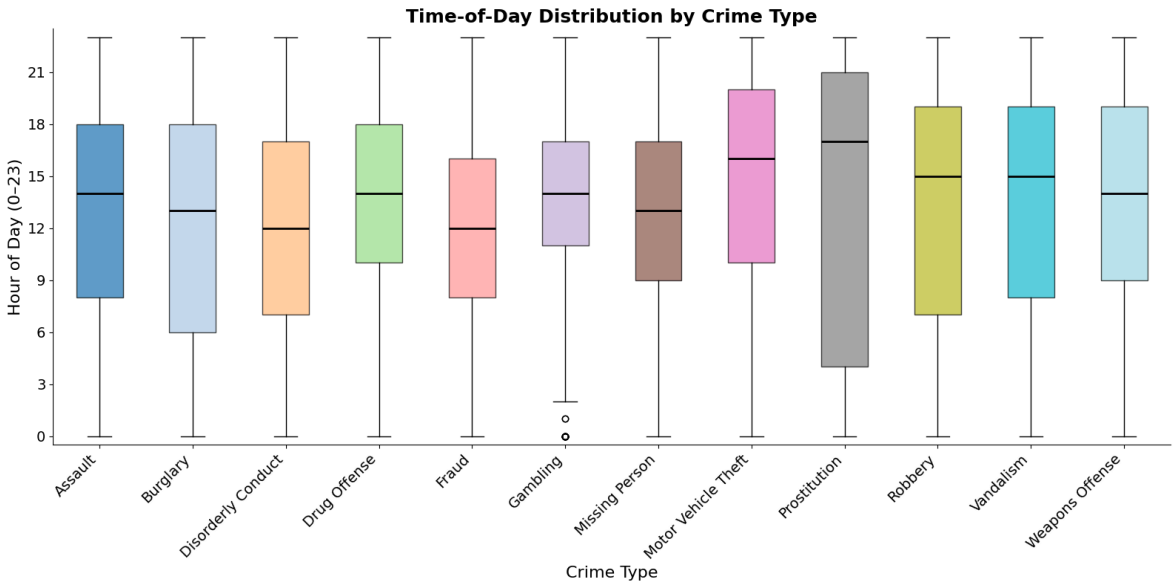
For Drug Offense, the QQ plot shows heavy tails — the lower tail curves below the line and the upper tail curves above it — indicating the distribution is more spread out than a normal, with pronounced clustering at both low and high latitudes. This reflects the well-known concentration of drug incidents in specific hotspots like the Tenderloin (northern-central SF) and the Mission, pulling the tails outward.

For Gambling, the deviation is more irregular, consistent with a multimodal distribution: incidents are not spread gradually across latitudes but are concentrated in a small number of distinct hotspot clusters, causing the QQ plot to follow a step-like pattern rather than a smooth curve. This makes sense given that gambling is a rare and, evidently, spatially concentrated crime.

In both cases the deviation from the diagonal tells us that crimes are not geographically uniform. They cluster into specific neighborhoods rather than occurring at random

locations across the city.

In [37]:
```
if 'hour' not in df.columns:
    df['hour'] = df['time'].str.split(':').str[0].astype(int)
crimes = sorted(df['crime'].unique())

# Prepare data for box plot
boxplot_data = [df[df['crime'] == c]['hour'].dropna().values for c in crimes]

fig, ax = plt.subplots(figsize=(16, 8))

bp = ax.boxplot(boxplot_data, patch_artist=True, notch=False,
                medianprops=dict(color='black', linewidth=2))

# Color each box
colors = plt.cm.tab20(np.linspace(0, 1, len(crimes)))
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)

ax.set_xticklabels([c.title() for c in crimes], fontsize=14, rotation=45, ha='ri
ax.set_xlabel('Crime Type', fontsize=16)
ax.set_ylabel('Hour of Day (0–23)', fontsize=16)
ax.set_title('Time-of-Day Distribution by Crime Type',
             fontsize=18, fontweight='bold')
ax.set_ylim(-0.5, 23.5)
ax.set_yticks(range(0, 24, 3))
ax.tick_params(axis='y', labelsize=14)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)


plt.tight_layout()
plt.show()
```



A fundamental limitation of the box plot for time-of-day data is the midnight wrap-around problem. The box plot has no way of knowing that hour 23 and hour 0 are adjacent on a clock. This means any crime with significant late-night and early morning activity will have its median and IQR pulled artificially towards the middle of the day, making it look more daytime-focused than it really is.

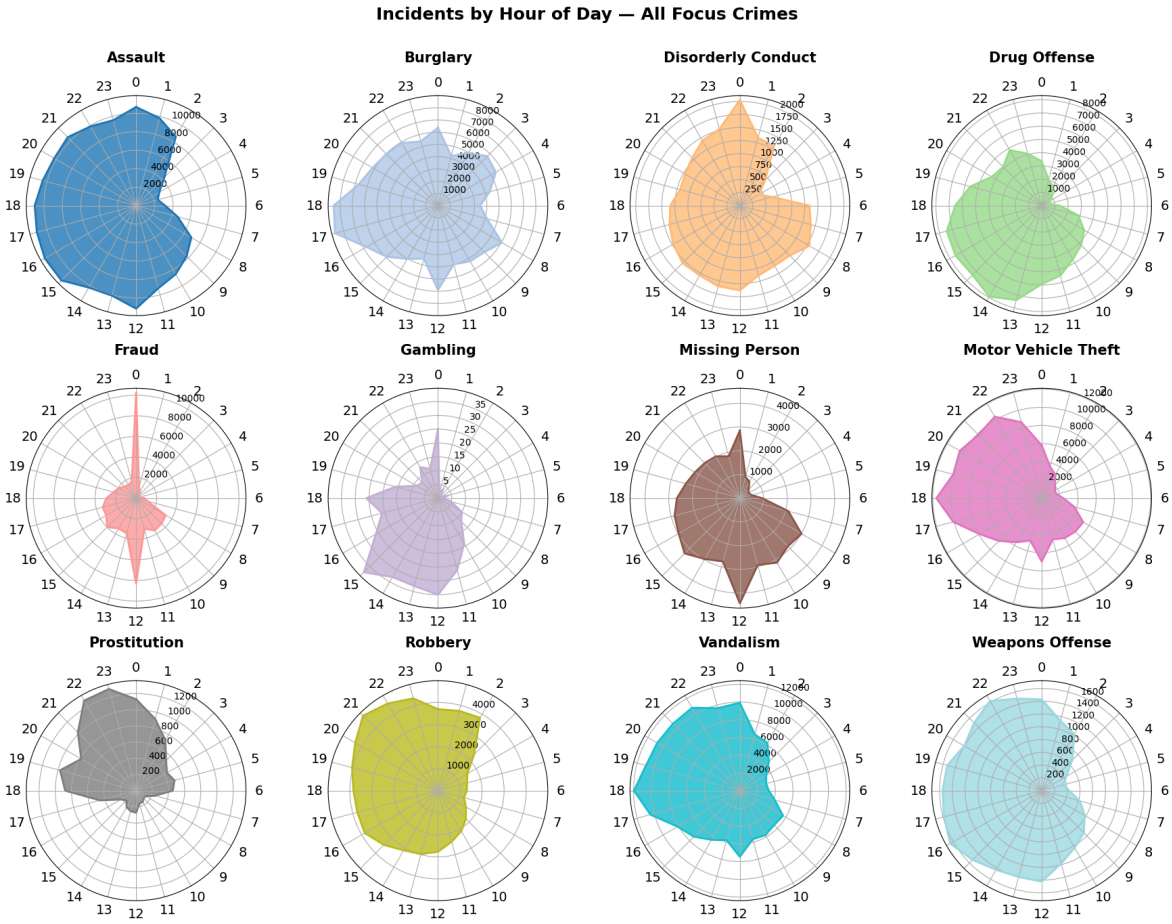
This directly affects prostitution: its apparent "shift towards late night" in the box plot is likely an understatement. The activity that wraps past midnight into the early hours is folded back to the low end of the scale, compressing what could be a very strong late-night/early-morning concentration into a wider, messier distribution.

Aside from this structural issue, most other crimes show a median around 13–14 with wide interquartile ranges spanning much of the day, indicating broadly distributed activity across waking hours rather than a sharp peak. This does not imply any particular distributional shape — it simply reflects that most crime types occur throughout the day without a dominant hour.

Gambling again stands out, with clustered outlier points rather than a continuous spread, consistent with the targeted enforcement operations suggested earlier.

To properly handle the wrap-around, polar plots are generated below of all the crimes in focus.

In [38]:
```
crimes_all = sorted(df['crime'].unique())
n_crimes_all = len(crimes_all)
ncols_p = 4
nrows_p = int(np.ceil(n_crimes_all / ncols_p))

color_palette_p = plt.cm.tab20(np.linspace(0, 1, n_crimes_all))

fig, axes = plt.subplots(nrows_p, ncols_p, figsize=(18, nrows_p * 4.5),
                         subplot_kw=dict(projection='polar'))
axes = axes.flatten()

theta_base = np.linspace(0, 2 * np.pi, 24, endpoint=False)

for i, crime in enumerate(crimes_all):
    ax = axes[i]
    hourly = df[df['crime'] == crime].groupby('hour').size().reindex(range(24),

    theta_c = np.append(theta_base, theta_base[0])
    counts_c = np.append(hourly.values, hourly.values[0])

    color = color_palette_p[i]
    ax.fill(theta_c, counts_c, alpha=0.8, color=color)
    ax.plot(theta_c, counts_c, color=color, linewidth=2)

    ax.set_xticks(np.linspace(0, 2 * np.pi, 24, endpoint=False))
    ax.set_xticklabels([f'{h}' for h in range(24)], fontsize=14)
    ax.set_theta_zero_location('N')
    ax.set_theta_direction(-1)
    ax.set_title(crime.title(), fontsize=15, fontweight='bold', pad=15)

for i in range(n_crimes_all, len(axes)):
    axes[i].set_visible(False)

fig.suptitle('Incidents by Hour of Day — All Focus Crimes',
             fontsize=18, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```



The polar plots reveal clear differences in when each crime type occurs. Most crimes — assault, burglary, robbery, vandalism — peak in the afternoon and evening, consistent with higher street activity during those hours.

Prostitution stands out with a strong late-night concentration between midnight and 04:00, confirming what the box plot underrepresented due to the wrap-around problem.

Drug offense peaks during daytime hours (roughly 10:00–18:00), which likely reflects both actual activity patterns and the hours during which police actively patrol and record incidents.

Gambling has very low overall counts and no clear daily rhythm, consistent with sporadic enforcement-driven reporting rather than a natural activity cycle.

Fraud is active broadly across business hours (08:00–18:00), but with a dominant spike at exactly 12:00. This sharp noon peak is most likely a recording artefact — incidents with an unknown or unspecified time are commonly defaulted to 12:00 in police databases. The underlying daytime pattern still makes sense given that fraud typically involves financial transactions and communications that occur during working hours.

Assignment 1.4: Spatial Power Law

Is crime spread evenly across San Francisco, or is it concentrated in a small number of hotspots? Let's find out.

Draws from: Week 4, Exercise 2.2.

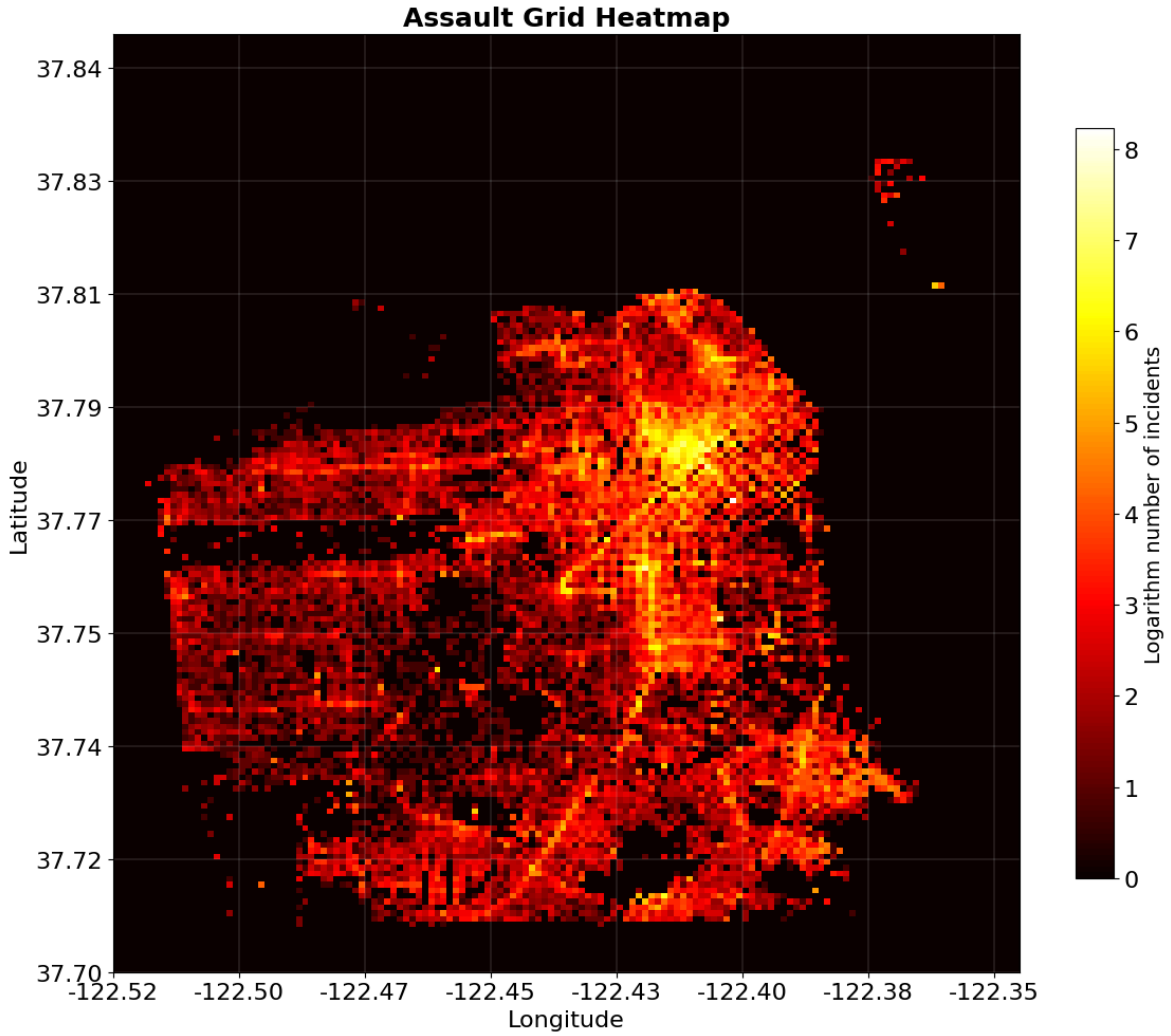
Use your **most common Personal Focus Crime** (by total incident count) for this exercise.

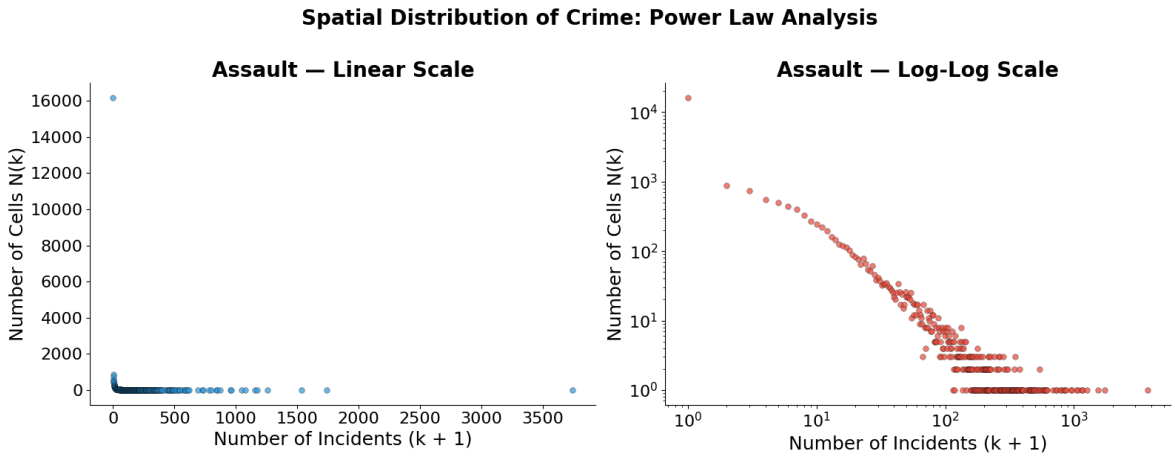
> - **Step 1**: Divide San Francisco into a grid of approximately $100\text{m} \times 100\text{m}$ cells using latitude and longitude. It is fine to ignore that the Earth isn't flat — the approximation is good enough for this purpose.
> - **Hint**: `np.histogram2d` works well here. Focus on points within the SF peninsula; filter out geographic outliers.
> - **Step 2**: Count the number of incidents of your chosen crime in each grid cell (use all years of data).
> - **Step 3**: Tally the distribution. Count how many cells have exactly k incidents — call this $N(k)$ — for $k = 0, 1, 2, \ldots$ up to the maximum.
> - **Step 4**: Plot $(k+1)$ vs $N(k)$ on **linear axes**.
> - **Step 5**: Plot $(k+1)$ vs $N(k)$ on **loglog axes**.
> - **Step 6**: Does your crime follow a power-law spatial distribution? How can you tell from the loglog plot?
> - **Step 7**: If crime is spatially concentrated in a small number of cells, what does that imply for how we interpret neighborhood-level crime statistics? Does the "average block" tell you anything useful?

In [39]:
```
# Recreate the 2D histogram for heatmap visualization
most_common = df['crime'].value_counts().idxmax()

sf_lat_min, sf_lat_max = 37.70, 37.85
sf_lon_min, sf_lon_max = -122.52, -122.35

crime_data = df[
    (df['crime'] == most_common) &
    (df['latitude'] >= sf_lat_min) & (df['latitude'] <= sf_lat_max) &
    (df['longitude'] >= sf_lon_min) & (df['longitude'] <= sf_lon_max)
].dropna(subset=['latitude', 'longitude'])

lat_bins = int((sf_lat_max - sf_lat_min) / 0.0009)
lon_bins = int((sf_lon_max - sf_lon_min) / 0.00118)

counts, lat_edges, lon_edges = np.histogram2d(
    crime_data['latitude'].values,
    crime_data['longitude'].values,
    bins=[lat_bins, lon_bins],
    range=[[sf_lat_min, sf_lat_max], [sf_lon_min, sf_lon_max]]
)

# Plot heatmap with grid overlay
fig, ax = plt.subplots(figsize=(12, 10))

# Use log scale for better visualization of hotspots (add 1 to avoid log(0))
im = ax.imshow(np.log1p(counts),
               extent=[sf_lon_min, sf_lon_max, sf_lat_min, sf_lat_max],
               origin='lower',
               cmap='hot',
```

```
                aspect='auto')

# Add grid lines with rounded tick labels
lon_ticks = lon_edges[::20]
lat_ticks = lat_edges[::20]
ax.set_xticks(lon_ticks)
ax.set_yticks(lat_ticks)
ax.set_xticklabels([f'{x:.2f}' for x in lon_ticks])
ax.set_yticklabels([f'{y:.2f}' for y in lat_ticks])
ax.grid(True, color='white', linewidth=0.3, alpha=0.5)

# Colorbar
cbar = plt.colorbar(im, ax=ax, shrink=0.8)
cbar.set_label('Logarithm number of incidents', fontsize=14)
cbar.ax.tick_params(labelsize=16)

ax.set_xlabel('Longitude', fontsize=16)
ax.set_ylabel('Latitude', fontsize=16)
ax.set_title(f'{most_common.title()} Grid Heatmap',
             fontsize=18, fontweight='bold')
ax.tick_params(axis='both', labelsize=16)

plt.tight_layout()
plt.show()

flat_counts = counts.flatten()
max_incidents = int(flat_counts.max())
k_values = np.arange(0, max_incidents + 1)
N_k = np.array([np.sum(flat_counts == k) for k in k_values])

# Only plot for k where N(k) > 0
mask = N_k > 0
k_plot = k_values[mask]
N_plot = N_k[mask]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Step 4: Linear axes
ax1 = axes[0]
ax1.scatter(k_plot + 1, N_plot, s=30, alpha=0.7, color='#3498db', edgecolor='bla
ax1.set_xlabel('Number of Incidents (k + 1)', fontsize=18)
ax1.set_ylabel('Number of Cells N(k)', fontsize=18)
ax1.set_title(f'{most_common.title()} — Linear Scale', fontsize=20, fontweight='
ax1.tick_params(axis='both', labelsize=16)
ax1.spines['top'].set_visible(False)
ax1.spines['right'].set_visible(False)

# Step 5: Log-log axes
ax2 = axes[1]
ax2.scatter(k_plot + 1, N_plot, s=30, alpha=0.7, color='#e74c3c', edgecolor='bla
ax2.set_xscale('log')
ax2.set_yscale('log')
ax2.set_xlabel('Number of Incidents (k + 1)', fontsize=18)
ax2.set_ylabel('Number of Cells N(k)', fontsize=18)
ax2.set_title(f'{most_common.title()} — Log-Log Scale', fontsize=20, fontweight=
ax2.tick_params(axis='both', labelsize=16)
ax2.spines['top'].set_visible(False)
ax2.spines['right'].set_visible(False)

fig.suptitle('Spatial Distribution of Crime: Power Law Analysis', fontsize=20, f
```

```
plt.tight_layout()
plt.show()
```





The top plot shows San Francisco divided into grid cells, displayed as a heatmap of incident counts for the most common crime type. The dominant hotspot is clearly Downtown San Francisco, confirming that crime is heavily concentrated in a small number of high-density areas. It is also worth noting that Treasure Island is captured in the data — the gap in the heatmap between the island and the main peninsula reflects the ocean between them.

It would be cool to overlay this image of San Francisco with a satelite image to get a proper picture.

Does the crime follow a power-law spatial distribution? Yes — the log-log plot is the key indicator here. A power law of the form $N(k) \propto k^{-\alpha}$ appears as a **straight line on a log-log plot**. Looking at the right panel, the points follow an approximately linear trend on the log-log axes across much of the range, which is consistent with a power-law distribution. The negative slope of roughly -2 to -3 (steeper in the tail) indicates that the number of cells $N(k)$ drops rapidly as the incident count k increases — most grid cells have very few incidents, while a small number of cells account for the vast majority of crimes.

This heavy-tail behaviour makes sense geographically: a handful of downtown blocks concentrate most of the recorded incidents, while large residential and commercial areas on the periphery see relatively little activity. Reporting an "average" incident count per cell would therefore be misleading — the distribution is far from uniform.

Assignment 1.5: Regression and Correlation

Which of your Personal Focus Crimes share the most similar weekly rhythm — and which are completely out of sync?

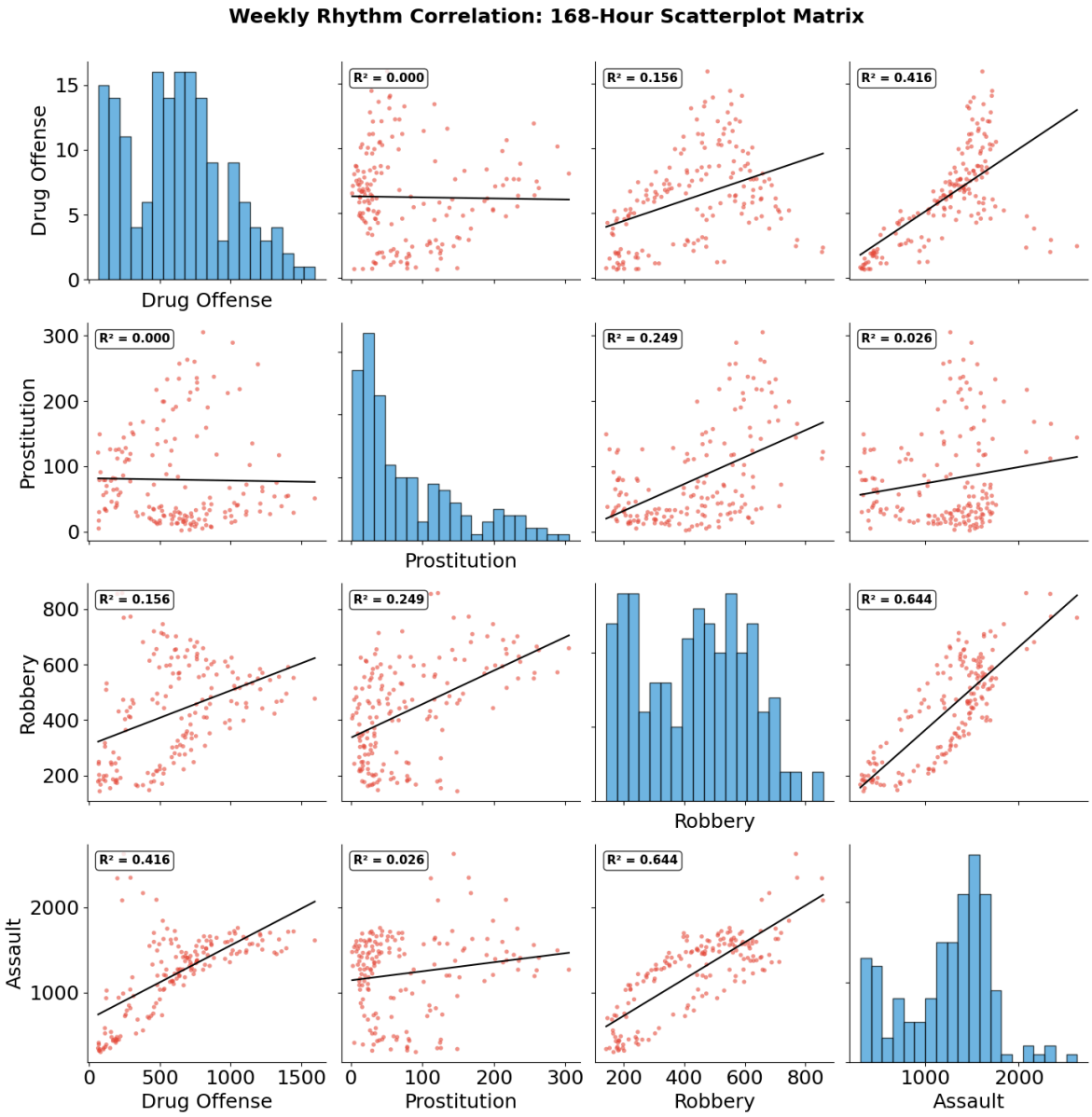
Draws from: Week 4, Exercises 3.1, 3.2, and 3.3.

Select **at least 4** of your Personal Focus Crimes for this exercise. More is fine - if you want to be closer to the actual weekly exercise.

> - For each selected crime type, compute the total number of incidents for each of the **168 hours of the week** (i.e., Monday 00:00–01:00, Monday 01:00–02:00, ..., Sunday 23:00–00:00). Each crime type will give you a vector of 168 values.
> - **Scatterplot matrix**: Create a grid of pairwise scatterplots — one panel per pair of crime types, with one crime on each axis and each of the 168 hours as a point. Label each panel clearly with the two crime types being compared.
> - **Linear regression**: Using the closed-form equations from Week 4 (not a library), fit a regression line $y = ax + b$ to each pair and add it to the relevant panel:
>
> $$a = \frac{\sum_i x_i y_i - N\langle x\rangle\langle y\rangle}{\sum_i x_i^2 - N\langle x\rangle^2}, \qquad b = \langle y\rangle - a\langle x\rangle$$
>
> - R^2: Compute R^2 for each pair and display the value on each panel.
> - Looking at your results: which pair of crimes is **most correlated**? Which is **least correlated**? Does the answer match your intuition? Why might two crime types share a weekly rhythm — or not?

In [40]:
```
# Selected crimes for correlation analysis
selected_crimes = ['drug offense', 'prostitution', 'robbery', 'assault']

# Parse datetime to get day of week (0=Monday, 6=Sunday)
df['datetime'] = pd.to_datetime(df['datetime'])
df['dayofweek'] = df['datetime'].dt.dayofweek

# Create 168-hour weekly vectors (7 days x 24 hours)
weekly_vectors = {}
for crime in selected_crimes:
    crime_df = df[df['crime'] == crime]
    # Group by (dayofweek, hour) and count
    weekly = crime_df.groupby(['dayofweek', 'hour']).size().reset_index(name='co
    # Create full 168-entry vector
    vector = np.zeros(168)
    for _, row in weekly.iterrows():
        idx = int(row['dayofweek']) * 24 + int(row['hour'])
        vector[idx] = row['count']
    weekly_vectors[crime] = vector

# Manual closed-form regression functions
def manual_regression(x, y):
    """Compute slope a, intercept b using closed-form equations."""
    N = len(x)
    x_mean = np.mean(x)
    y_mean = np.mean(y)

    # a = (sum(xi*yi) - N*<x>*<y>) / (sum(xi^2) - N*<x>^2)
    numerator = np.sum(x * y) - N * x_mean * y_mean
    denominator = np.sum(x**2) - N * x_mean**2

    if denominator == 0:
        return 0, y_mean, 0

    a = numerator / denominator
    b = y_mean - a * x_mean

    # R² = 1 - SS_res / SS_tot
    y_pred = a * x + b
    ss_res = np.sum((y - y_pred)**2)
    ss_tot = np.sum((y - y_mean)**2)
    r_squared = 1 - ss_res / ss_tot if ss_tot > 0 else 0

    return a, b, r_squared

# Create 4x4 scatterplot matrix
n = len(selected_crimes)
fig, axes = plt.subplots(n, n, figsize=(14, 14))

for i, crime_y in enumerate(selected_crimes):
    for j, crime_x in enumerate(selected_crimes):
        ax = axes[i, j]
        x = weekly_vectors[crime_x]
        y = weekly_vectors[crime_y]

        if i == j:
            # Diagonal: histogram of the crime's weekly distribution
            ax.hist(y, bins=20, color='#3498db', alpha=0.7, edgecolor='black')
            ax.set_xlabel(crime_x.title(), fontsize=18)
```

```
        else:
            # Off-diagonal: scatter + regression
            ax.scatter(x, y, s=15, alpha=0.6, color='#e74c3c', edgecolor='none')

            # Compute regression
            a, b, r_sq = manual_regression(x, y)

            # Plot regression line
            x_line = np.linspace(x.min(), x.max(), 100)
            y_line = a * x_line + b
            ax.plot(x_line, y_line, color='black', linewidth=1.5)

            # Display R²
            ax.text(0.05, 0.95, f'R² = {r_sq:.3f}', transform=ax.transAxes,
                    fontsize=11, fontweight='bold', va='top',
                    bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

        # Labels only on edges
        if i == n - 1:
            ax.set_xlabel(crime_x.title(), fontsize=18)
        else:
            ax.set_xticklabels([])

        if j == 0:
            ax.set_ylabel(crime_y.title(), fontsize=18)
        else:
            ax.set_yticklabels([])

        ax.tick_params(axis='both', labelsize=18)
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)

fig.suptitle('Weekly Rhythm Correlation: 168-Hour Scatterplot Matrix',
             fontsize=18, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

# Print R² summary
print("\nR² Summary (pairwise correlations):")
print("-" * 45)
for i, crime_y in enumerate(selected_crimes):
    for j, crime_x in enumerate(selected_crimes):
        if i > j:  # Lower triangle only
            x = weekly_vectors[crime_x]
            y = weekly_vectors[crime_y]
            _, _, r_sq = manual_regression(x, y)
            print(f"{crime_x.title():15} vs {crime_y.title():15}: R² = {r_sq:.4f
```




```
R² Summary (pairwise correlations):
--------------------------------------------
Drug Offense     vs Prostitution    : R² = 0.0003
Drug Offense     vs Robbery         : R² = 0.1560
Prostitution     vs Robbery         : R² = 0.2491
Drug Offense     vs Assault         : R² = 0.4162
Prostitution     vs Assault         : R² = 0.0264
Robbery          vs Assault         : R² = 0.6437
```

Looking at the R^2 values, Assault and Drug Offense show the highest correlation. This makes sense as both are crimes that follow similar urban activity patterns — they peak during afternoon/evening hours when more people are on the streets, and both are more common in high-density areas like downtown and the Tenderloin, Tenderloin as mentioned earlier has endured a lot of issues with drugs and gangs.

- Crimes tied to street activity (assault, drug offense, prostitution) share similar patterns because they depend on people being out and about
- Business-hour crimes would correlate differently than nightlife-related crimes
- Prostitution has a distinct late-night pattern that partially overlaps with drug activity in areas like the Tenderloin

The scatterplot matrix reveals that crimes with similar "opportunity structures" (when/where they can occur) tend to have more correlated weekly rhythms.

And whilst a lot can be deduced from this data it is always worth remembering thatcorrelations can be spurrious sometimes and maybe be a unlucky coincidence. My favorite website for instance highlights the importance of domain knowledge when drawing conclusions based on correlations.

https://www.tylervigen.com/spurious-correlations

Contributions

All 3 group members worked in unison and contributed equally to this assignment.